# PostgreSQL CDC 수집과 변경 이벤트 검증

2회차 · 120분
원천 설정부터 Kafka 메시지, Bronze 행, Parquet 파일까지

강사 맥미니 시연 · 학생 SQL·이벤트 예측 · 개인 환경 복습

# 오늘의 120분

| 시간 | 진행 | 학생 활동 |
|---|---|---|
| 00:00-00:10 | 복습 진행 공유 | 해본 것과 막힌 점 |
| 00:10-00:30 | 지난 시간의 저장 결과 | Bronze 3이력 → Parquet |
| 00:30-00:50 | PostgreSQL·Debezium 설정 | 발행 범위 · 이전 값 · 소비 위치 |
| 00:50-00:55 | 휴식 | 5분 |
| 00:55-01:15 | 스냅샷과 이벤트 파서 | r · c/u/d · tombstone |
| 01:15-01:35 | 새 환불 예측과 실행 | Kafka UI → Bronze · 실제 파싱 |
| 01:35-01:50 | 수집 구간 진단 | 어디까지 도착했는가 |
| 01:50-02:00 | 정리와 복습 | 개인 실행 · 다음 회차 |

# 복습하면서 어디까지 해보셨나요?

## 해본 것

환경을 띄웠는지,
어떤 명령을 실행했는지

## 확인한 결과

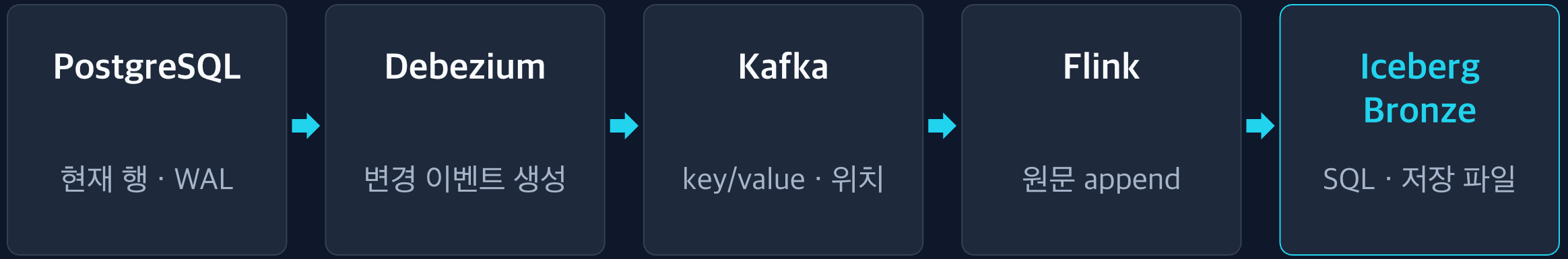
예상했던 값과
실제 결과가 같았는지

## 막힌 부분

오류 메시지와
다음에 시도할 것

**SQL이나 실행 화면을 보면서 편하게 나눕니다**

# 오늘 확인할 수집 경로




Spark SQL은 Bronze를 읽는 데 사용합니다
Silver·Gold 변환과 dbt 구현은 후속 회차입니다

# 삭제한 환불은 어디에 남을까요?

| 단계 | 원천 PostgreSQL | Kafka value / Bronze payload |
|---|---|---|
| INSERT | 1행 · 20,000원 | c · after.amount = 20000 |
| UPDATE | 1행 · 30,000원 | u · before 20000 → after 30000 |
| DELETE | 0행 | d · before 30000 → after null |

**같은 PK의 c/u/d는 서로 다른 변경 이력입니다**

# Bronze에서 같은 환불의 세 이력 조회

Spark SQL · lake.bronze.cdc_events

```
SELECT get_json_object(payload, '$.op') AS op,
       get_json_object(payload, '$.before.amount') AS before_amount,
       get_json_object(payload, '$.after.amount') AS after_amount,
       topic, partition_id, kafka_offset, _file AS parquet_file
FROM lake.bronze.cdc_events
WHERE topic = 'cdc.public.refunds'
  AND (get_json_object(payload, '$.after.refund_id') = '<환불_ID>'
    OR get_json_object(payload, '$.before.refund_id') = '<환불_ID>')
ORDER BY partition_id, kafka_offset;
```

DELETE는 before의 PK도 찾습니다

# 조회 결과를 이렇게 대조합니다

| 확인할 것 | 판단할 근거 |
| --- | --- |
| 같은 환불인가 | before / after의 refund_id |
| 변경 내용이 맞는가 | op와 20,000 → 30,000 → 삭제 |
| 어느 메시지인가 | topic + partition_id + kafka_offset |
| 어느 파일에 있는가 | parquet_file |

**오늘의 관찰은 이번 키의 결과입니다. 전체 데이터의 무누락 보장은 아닙니다**

# 현재 테이블이 참조하는 파일 확인

Spark SQL · Iceberg files 메타데이터 테이블

```
SELECT file_path, file_format, record_count, file_size_in_bytes
FROM lake.bronze.cdc_events.files
WHERE content = 0
ORDER BY file_path LIMIT 10;
```

files의 record_count는 파일 전체 행 수입니다. 이번 환불 행 수와 구분합니다

# Parquet 파일의 내용까지 확인

**Bronze SQL**
대상 행 + 파일 경로

→

**files 메타데이터**
현재 파일 참조

→

**MinIO 데이터 파일**
실제 Parquet 읽기

→

**내용 대조**
payload · 위치

metadata.json이나 파일 이름을 보는 데서 끝내지 않습니다

# 파일 확인에서 구분할 것

### 변경 이벤트

c/u/d 각각이
Bronze의 한 이력

### 데이터 파일

여러 키·이벤트가
같이 들어갈 수 있음

### Iceberg snapshot

테이블이 참조하는
파일 집합의 버전

**DML 한 번 = 파일 한 개 = snapshot 한 개로 가정하지 않습니다**

# PostgreSQL에서 확인할 수집 조건

## wal_level

WAL에 남길 변경 정보 수준
CDC 필요 조건: logical

## publication

발행할 테이블과
변경 종류의 범위

## replica identity

UPDATE·DELETE에서
이전 행을 식별할 정보

## replication slot

소비 진행 위치와
필요한 WAL 보존

**이름이 비슷해도 역할은 다릅니다**

# 조건 1 · wal_level = logical 확인

PostgreSQL · app · 읽기 전용 조회

```
-- 현재 PostgreSQL의 WAL 기록 수준 확인
SHOW wal_level;

-- [참고] wal_level 3단계 비교
-- minimal : 비정상 종료 복구에 필요한 최소 정보만 기록
-- replica : 물리 복제 · 시점 복구(PITR)용 기록 (PG 기본값)
-- logical : 행 단위 변경(CDC)을 해독할 논리 정보까지 추가 기록
```

SHOW는 현재 값을 조회하는 명령입니다 · 이번 실습의 기대값은 logical
wal_level은 논리 디코딩에 필요한 기록 수준, publication은 발행 범위를 정합니다

# publication: 발행할 테이블과 변경 종류

## 발행 대상 테이블

cdc_lab publication
public.orders · 주문
public.payments · 결제
public.refunds · 환불

## 발행할 변경 종류

INSERT · UPDATE · DELETE · TRUNCATE
변경 종류별 발행 여부 선택
현재 cdc_lab은 네 가지 모두 허용
오늘 확인할 변경은 생성·수정·삭제

**publication은 어떤 테이블의 어떤 변경을 발행할지 정하는 PostgreSQL 설정입니다**
**설정만으로 전송이 시작되지는 않으며, Debezium 같은 소비자가 연결해 읽어야 합니다**

# cdc_lab의 발행 범위 확인

PostgreSQL · app · 읽기 전용 조회

```
-- 1) 어떤 테이블을 발행하는가? (3행 확인)
SELECT schemaname, tablename
FROM pg_publication_tables
WHERE pubname = 'cdc_lab';

-- 2) 어떤 변경 종류를 허용하는가? (4개 플래그 확인)
SELECT pubname, pubinsert, pubupdate, pubdelete, pubtruncate
FROM pg_publication
WHERE pubname = 'cdc_lab';
```

pg_publication_tables: 발행 대상 테이블 · pg_publication: 허용 변경 종류
현재 예제는 public 스키마의 세 테이블, 네 가지 변경 플래그는 모두 true입니다

# 이전 금액은 어떤 설정으로 관찰할까요?

PostgreSQL · 현재 설정 조회

```
SELECT c.relname, c.relreplident
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public'
  AND c.relname IN ('orders', 'payments', 'refunds');
```

이번 실습은 relreplident = f, REPLICA IDENTITY FULL입니다

# slot의 두 위치와 WAL 보존

| 필드 | 읽을 의미 |
| --- | --- |
| active | 현재 slot을 스트리밍하는 소비자가 있는가 |
| restart_lsn | slot이 아직 필요로 할 수 있는 가장 오래된 WAL 위치 |
| confirmed_flush_lsn | 논리 slot 소비자가 수신을 확인한 위치 |
| wal_status (참고) | 공식 뷰의 WAL 가용 상태 |

**confirmed_flush_lsn은 Iceberg 적재 완료 위치가 아닙니다**

# Debezium이 Kafka 이벤트를 만드는 과정




Task·Converter·Producer는 Kafka Connect worker 안에서 동작합니다
초기 snapshot은 기존 행을 r로, 이후 스트리밍은 행 변경을 c/u/d로 전달합니다

# Debezium 설정과 실제 수집 범위

실제 설정 중 학습에 필요한 항목만 표시

```
{
  "plugin.name": "pgoutput",
  "slot.name": "cdc_lab",
  "publication.name": "cdc_lab",
  "publication.autocreate.mode": "disabled",
  "table.include.list": "public.orders,public.payments,public.refunds",
  "topic.prefix": "cdc",
  "snapshot.mode": "initial",
  "tombstones.on.delete": "true"
}
```

publication · include list · 실제 이벤트를 함께 확인합니다

# 5분 쉬고 이어가겠습니다

돌아오면 스냅샷 r과 작은 파서로 이벤트를 해석합니다

# 초기 스냅샷 r과 이후 c/u/d

| 구분 | 무엇을 나타내는가 | 이번 시간의 확인 |
| --- | --- | --- |
| r | 스냅샷에서 읽은 현재 행 | 보존된 실제 표본 또는 표시된 fixture |
| c | 수집 시작 이후 생성 이벤트 | 새 환불 20,000원 |
| u | 기존 행의 변경 이벤트 | 같은 환불 30,000원으로 정정 |
| d | 물리 삭제 이벤트 | before 값과 after=null |

**snapshot.mode=initial은 매 재시작마다 전체 스냅샷을 뜬다는 뜻이 아닙니다**

# 작은 파서의 입력과 출력

### 입력

Kafka value 원문
+ key · topic · partition · offset

### 해석

JSON 구조와 op 확인
DELETE는 before에서 PK

### 출력

변경 요약 또는
tombstone · 오류 분류

**원문 적재 파이프라인을 바꾸지 않는 독립적인 해석 도구입니다**

# 변경 종류에 따라 행을 선택하기

Python · event_parser.py의 핵심 분기

```
# 키 검증을 통과한 뒤의 핵심 분기만 발췌했습니다.
if raw_value is None:
    return {'kind': 'tombstone'}

value = json.loads(raw_value)
if value is None:
    return invalid('invalid_json_null')
op = value['op']
row = value.get('before') if op == 'd' else value.get('after')
# 전체 검증과 PK 선택은 event_parser.py에서 확인합니다.
```

**왜 DELETE에서 after를 선택하면 안 될까요?**

# 파서 결과를 입력별로 확인

| 입력 | 확인할 것 |
| --- | --- |
| 생성 c | after에서 PK와 현재 값 |
| 삭제 d | before에서 PK, after=null |
| wire value=null | tombstone으로 구분 |
| JSON 텍스트 null | invalid_json_null |
| 나머지 입력 | r·u·오류 검증은 복습에서 확인 |

**이어서 새 환불을 실행하고 실제 메시지와 비교합니다**

# Kafka UI에서 같은 변경 확인

### key

Key 탭의 refund_id
같은 원천 행을 식별

### value

Value 탭의 변경 내용
op · before · after

### 수집 위치

topic · partition · offset
Bronze에서 같은 위치 대조

**http://localhost:28082 · cdc.public.refunds · Messages**
**새 환불 ID로 Search 후 Refresh · 행의 +로 Key / Value 확인**

# DELETE와 tombstone은 다릅니다

## 삭제 이벤트

value는 JSON 객체
op=d
before=삭제 전 행
after=null

## tombstone

key는 같은 환불 ID
wire value=null
UI: 빈 Value · Size 0 Bytes
이번 Bronze에서는 제외

**빈 Value는 CLI의 value:null과 대조해 확인합니다**
**Kafka 메시지 수와 Bronze 행 수는 다를 수 있습니다**

# 새 환불을 실행하기 전에 예측

| 변경 | 원천 행 수 | before.amount | after.amount |
|---|---|---|---|
| INSERT 20,000 | ? | ? | ? |
| UPDATE 30,000 | ? | ? | ? |
| DELETE | ? | ? | ? |

**예측을 적고 실제 key · op · 값 · 위치와 비교합니다**

# 원천 변경부터 Bronze 조회까지 대조

**원천 변경**

환불 c → u → d

**원천 SELECT**

현재 행 확인

**Kafka UI**

새 환불 ID · Refresh
Key / Value · offset

**Bronze 확인**

같은 위치와 내용

**UI와 Bronze를 topic · partition · offset으로 연결합니다**

# 수집이 안 보일 때 확인할 구간

| 관찰 | 먼저 확인할 것 |
| --- | --- |
| 원천만 바뀜 | commit · publication · include list · connector/task |
| Kafka에는 있고 Bronze에는 없음 | Flink 입력 · 처리 · checkpoint · sink commit |
| Bronze에 c와 u만 보임 | after-only 조건 · before PK · 실제 key와 위치 |
| 한 파일에 c만 보임 | 현재 files 목록 · 같은 키의 전체 Bronze 조회 |

**“안 된다” 대신 마지막으로 확인한 지점을 말합니다**

# 오늘 확인한 증거

## 수집 설정

publication · slot · FULL
Debezium include list

## 변경 이벤트

같은 환불의 c/u/d
key · 값 · 전달 위치

## 저장 결과

Bronze 세 이력
현재 Parquet 내용

**설정이 맞는지와 결과가 맞는지를 함께 확인합니다**

# 복습과 다음 회차

## 개인 환경 복습

공통 예제 실행
SQL·파서 결과·막힌 점 정리

## 제출과 공유

submission.md + SQL
진행한 과정을 동료와 공유

## 3회차

Flink → Bronze 적재 심화
개인 도메인·테이블 설계 확정

**제출 방법·기한은 별도 안내를 따릅니다**

# 공식 문서와 실습 자료

| 자료 | 확인할 내용 |
| --- | --- |
| PostgreSQL 17 | Logical decoding · publication · replication slots |
| Debezium 3.2 | PostgreSQL connector · snapshot · change events |
| Apache Kafka 3.9 | Partition ordering · record metadata |
| Apache Iceberg 1.10.0 | 파일 조회 · Parquet 디코드 · Flink writes |
| Apache Spark 3.5.6 | Parquet 읽기 (별도 참고) |
| GitHub | kim-dabin/cdc-lakehouse-study · labs/session02 |

공식 링크는 labs/session02/references.md에 있습니다